

Connectedness

Lambda calculus, recursion, probability, and logic

Cem Bozsahin

LING487 · Fall 2026

Four formalisms ask the same first question

Question

What must match before one expression can connect to another?

- Lambda calculus: a function must accept the argument supplied.
- Recursion: an output must re-enter the same pattern as an admissible input.
- Probability: events must inhabit a specified sample space.
- Logic: conclusions must follow under explicit rules of inference.

MOLR §§1.4–1.7

Where we are

1. Lambda abstraction and application
2. Recursion
3. Probability
4. Logic

A lambda term turns a dependency into an object

$$x + 1 \rightsquigarrow \lambda x. x + 1$$

- The **body** says what is done.
- The lambda binds the placeholder on which the body depends.
- Application supplies a value: $(\lambda x. x + 1)2$.

Take-away

Look to the body for the result; look to the lambda for what the expression awaits.

Beta reduction is disciplined substitution

$$(\lambda x.M)N \longrightarrow_{\beta} M[x/N]$$

$$(\lambda x.x^2)3 \longrightarrow_{\beta} 3^2 = 9$$

Avoid capture

Only free occurrences of x in M are replaced; bound variables may need renaming first.

Guided analysis

Reduce $(\lambda x.\lambda y.x - y)2\ 5$. Which argument is consumed first?

Currying replaces one tupled input with a sequence

$$\begin{array}{ccc} f: A \times B \rightarrow C & \iff & f: A \rightarrow C^B \\ \lambda \langle x, y \rangle . M & \iff & \lambda x. \lambda y. M \end{array}$$

- Currying changes how arguments are supplied, not the final result.
- Each abstraction exposes one stage of connectedness.
- Linguistic categories likewise encode what remains to be combined.

MOLR §1.4

Application can be factored through composition

Define the raising combinator

$$\mathbf{T} = \lambda x. \lambda f. fx, \quad \mathbf{T}a = \lambda f. fa.$$

Then an argument can first become functional and subsequently compose.

Conceptual point

Application and composition need not be rival mechanisms. Raising can expose their common connectedness.

Guided analysis

Raise 2, apply the result to $\lambda x.x^2$, and beta-reduce.

MOLR §1.4.5

Where we are

1. Lambda abstraction and application
2. Recursion
3. Probability
4. Logic

Recursion is not the same thing as repetition

$$n! = \begin{cases} 1 & n = 0 \\ n(n-1)! & n > 0 \end{cases} \quad n! = 1 \times 2 \times \cdots \times n$$

- The first description is name-recursive.
- The second describes the same values without self-reference.
- Repetition may be finite, iterative, recursive, or all three.

Question

What property of the result is explained by the recursive version?

Linguistic embedding can be arbitrarily deep in description

I think [you think [she thinks [I like candies]]].

- A clause can contain another object of a relevantly similar kind.
- A finite grammar can describe an arbitrarily large family of structures.
- Human performance limits do not by themselves define grammatical structure.

Guided analysis

Give a nonrecursive paraphrase of the example. What structural generalization does it lose?

MOLR §1.5

Where we are

1. Lambda abstraction and application
2. Recursion
3. Probability
4. Logic

Probability requires a sample space before a number

$$P : A \rightarrow [0, 1], \quad P(\Omega) = 1$$

- Ω states the outcomes treated as possible.
- An event is a subset of Ω .
- A probability is therefore conditional on an abstraction of possibilities.

Guided analysis

If the sample space is “tokens in a corpus,” what changes when the corpus changes?

MOLR §1.6

Conditional probability reverses only with evidence

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)} \quad (P(B) > 0)$$

$P(A \mid B) \neq P(B \mid A)$ in general.

- Joint probability concerns the intersection.
- Conditioning restricts the space to cases in B .
- Statistical association alone does not establish a linguistic dependency.

Where we are

1. Lambda abstraction and application
2. Recursion
3. Probability
4. Logic

Validity concerns the connection from premises to conclusion

$A, A \rightarrow B \vdash B$ (modus ponens)

$A \rightarrow B, \neg B \vdash \neg A$ (modus tollens)

- Validity does not guarantee that the premises are true.
- Truth does not guarantee that an argument is valid.
- A formal proof makes every licensed transition inspectable.

MOLR §1.7

Quantifiers connect predicates with domains

$$\forall x [child(x) \rightarrow slept(x)]$$

- $\forall$ and $\exists$ range over a chosen domain.
- The same sentence can receive distinct scope structures.
- A formula is not an interpretation until its vocabulary and domain are fixed.

Guided analysis

Formalize “Some child did not sleep.” Contrast it with “Not every child slept.”

Third hour: one connectedness, four notations

For each expression, identify the objects and the condition that licenses the next step.

- ① $(\lambda x.x + 1)4$
- ② $f(n) = nf(n - 1)$
- ③ $P(A \mid B)$
- ④ $A, A \rightarrow B \vdash B$

Guided analysis

Write one sentence explaining why none of the four operations is “mere symbol manipulation.”

Week 2 synthesis

- ① Lambda abstraction exposes dependencies; application satisfies them.
- ② Currying stages a multi-argument dependency.
- ③ Recursion feeds a structurally appropriate result back into a definition.
- ④ Probability depends on a declared space of possibilities.
- ⑤ Logic makes admissible inference explicit.

Question

Quiz 1 asks for one small construction or comparison from this vocabulary.